

Chapter 3: Reinforcement Learning

Takashi Tanaka

Contents

3.1 Problem Setup	41
3.2 Model-based RL: Value iteration	42
3.3 Simulator-Driven RL: Monte Carlo Method	46
3.3.1 Q-Function	46
3.3.2 Monte Carlo ES: Discounted Problems	47
3.3.3 Monte Carlo ES: Undiscounted Problems	48
3.4 Data-driven RL: Temporal difference method	49
3.4.1 Stochastic approximation	49
3.4.2 Convergence of Q-Learning	50
3.4.3 Monte Carlo vs. TD Learning	52

3.1 Problem Setup

Consider a time-invariant Markov Decision Process (MDP) over discrete time steps $t = 0, 1, 2, \dots$ defined by the tuple $(\mathcal{X}, \mathcal{U}, \tau, C, \gamma)$, where $\mathcal{X}$ is a finite state space, $\mathcal{U}$ is a finite action space, $\tau(x_{t+1}|x_t, u_t)$ is the transition probability, $C(x_t, u_t)$ is the stage-wise cost, and $\gamma \in (0, 1)$ is the discount factor. In Section 2.2.3, we introduced an iterative policy synthesis algorithm (Algorithm 3) for the discounted-cost problem:

$$\min_{\mu_{0:\infty}} \sum_{t=0}^{\infty} \gamma^t \mathbb{E}^{\mu_{0:\infty}} [C(x_t, u_t)]. \quad (3.1)$$

In this chapter, we further study the convergence properties of iterative policy synthesis algorithms in the model-based, simulator-driven, and data-driven contexts. For model-based scenarios, we analyze the convergence of the *value iteration* algorithm. Value iteration is one of the most basic forms of reinforcement learning (RL), and can be viewed as a natural extension of the dynamic programming arguments discussed in Section 2.2.3.

However, the main utility of RL algorithms lies in their applicability to simulator-driven and data-driven settings. Therefore, our theoretical interest focuses on analyzing their performance in such contexts. In what follows, we study the convergence of RL algorithms under the simplifying assumption that both $\mathcal{X}$ and $\mathcal{U}$ are finite. In this case, both the value function and the optimal policy can be stored in a lookup table, and RL algorithms under this assumption are referred to as *tabular methods*.

3.2 Model-based RL: Value iteration

For a time-invariant MDP $(\mathcal{X}, \mathcal{U}, \tau, C, \gamma)$ without an absorbing state, recall that the optimal policy solving (3.1) is expressed as

$$u_t^* = \mu(x_t) = \arg \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J^*(x') \tau(x'|x, u) \right\} \quad (3.2)$$

where $J^*(\cdot)$ is obtained as the limit of the following recursion:

$$J_{k+1}(x) = \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J_k(x') \tau(x'|x, u) \right\}. \quad (3.3)$$

Recall that the convergence of J_k to J^* and the optimality of u_t^* is granted by converting (3.1) into an equivalent variable end-time problem, and applying Theorem 2.3. Let us generalize this argument to the problems with variable end-times and discounted costs.

Example 3.1 (*Gridworld*) Consider a simple 5×5 gridworld shown in Figure 3.1. There are two absorbing states $\mathcal{X}_{abs} = \{x_{pit}, x_{goal}\}$, and 23 regular (non-absorbing) states. The set of regular states is denoted by $\mathcal{X}_{reg}$. The process starts at $x_0 \in \mathcal{X}_{reg}$ and terminates at a random exit time

$$T_f = \min\{t > 0 : x_t \in \mathcal{X}_{abs}\}. \quad (3.4)$$

Entering into x_{pit} incurs the exit cost of $C^{exit}(x_{pit}) = c_{pit} > 0$, while entering into x_{goal} incurs $C^{goal}(x_{goal}) = c_{goal} < 0$. Four possible actions $\mathcal{U} = \{\text{“Left”}, \text{“Right”}, \text{“Up”}, \text{“Down”}\}$ are available at each regular state. The state transition probability is defined by the following rules:

	1	2	3	4	5
1					x_{goal}
2					
3					
4					
5					x_{pit}

Figure 3.1: 5×5 gridworld.

- For each regular state x ,

$$\tau(x'|x, u) = \begin{cases} 1 - p & \text{if } x' \text{ is the intended next state} \\ p/2 & \text{if } x' \text{ is one of the two lateral states} \\ 0 & \text{otherwise.} \end{cases} \quad (3.5)$$

That is, the traveling agent “slips” with probability p .

- If the move determined by (3.5) points outside the grid, the agent stays in the same state. See Figure 3.2.

We also assume that the step cost is $C(x, u) = c_{\text{step}} > 0$ for all regular states $x \in \mathcal{X}_{\text{reg}}$ and $u \in \mathcal{U}$.

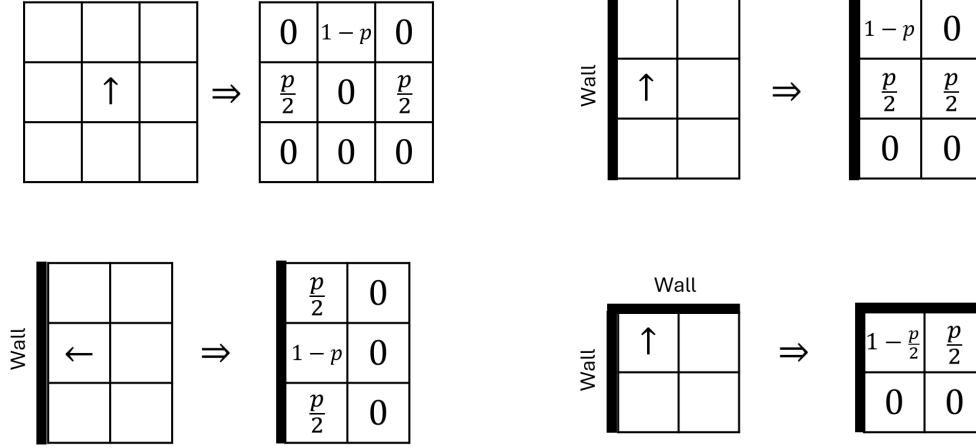


Figure 3.2: Grid transition rule.

Given an MDP setup such as Example 3.1, our goal is to find a policy that solves the following variable end-time discounted-cost problem:

$$\min_{\mu_{0:\infty}} \sum_{t=0}^{T_f-1} \gamma^t \mathbb{E}[C(x_t, u_t)] + \gamma^{T_f} \mathbb{E}[C^{\text{exit}}(x_{T_f})]. \quad (3.6)$$

Partitioning the state space as $\mathcal{X} = \mathcal{X}_{\text{reg}} \cup \mathcal{X}_{\text{abs}}$, the value iteration algorithm for (3.6) is obtained as follows: First, we convert (3.6) into an equivalent undiscounted variable end-time problem. This is achieved by introducing an additional absorbing state to account for the discount factor and by defining a new transition kernel $\bar{\tau}$ in the same manner as in Section 2.2.3. Next, we apply Algorithm 3, whose correctness is guaranteed by Theorem 2.3. Finally, we rewrite the algorithm using the original transition kernel τ .

From this construction, it follows that the following value iteration correctly computes the value function of (3.6): First, initialize $J_0 : \mathcal{X} \rightarrow \mathbb{R}$ by setting $J_0(x) = C^{\text{exit}}(x)$ for $x \in \mathcal{X}_{\text{abs}}$, and assigning arbitrary values for $x \in \mathcal{X}_{\text{reg}}$. Then, for $k = 0, 1, 2, \dots$, repeat:

$$J_{k+1}(x) = C^{\text{exit}}(x), \quad \forall x \in \mathcal{X}_{\text{abs}}, \quad (3.7a)$$

$$J_{k+1}(x) = \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J_k(x') \tau(x'|x, u) \right\}, \quad \forall x \in \mathcal{X}_{\text{reg}}. \quad (3.7b)$$

For convenience, we denote this iteration compactly as $J_{k+1} = TJ_k$, where the operator T is defined by

$$(TJ)(x) = \begin{cases} C^{\text{exit}}(x), & x \in \mathcal{X}_{\text{abs}}, \\ \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J(x') \tau(x'|x, u) \right\}, & x \in \mathcal{X}_{\text{reg}}. \end{cases} \quad (3.8)$$

We now analyze the convergence rate of the value iteration $J_{k+1} = TJ_k$. Define the maximum norm on $\mathcal{X}$ by

$$\|J\|_\infty := \max_{x \in \mathcal{X}} |J(x)|.$$

The next lemma establishes the contraction property of the operator T :

Lemma 3.1 *For any pair of functions $J : \mathcal{X} \rightarrow \mathbb{R}$ and $\bar{J} : \mathcal{X} \rightarrow \mathbb{R}$, we have*

$$\|TJ - T\bar{J}\|_\infty \leq \gamma \|J - \bar{J}\|_\infty. \quad (3.9)$$

Proof: If $d = \|J - \bar{J}\|_\infty$, then

$$J(x) - d \leq \bar{J}(x) \leq J(x) + d \quad \forall x \in \mathcal{X}. \quad (3.10)$$

From the definition (3.8) of T , we have

$$(TJ)(x) = (T\bar{J})(x) = C^{\text{exit}}(x) \quad \forall x \in \mathcal{X}_{\text{abs}}. \quad (3.11)$$

Therefore,

$$(TJ)(x) - (T\bar{J})(x) = 0 \quad \forall x \in \mathcal{X}_{\text{abs}}. \quad (3.12)$$

For each $x \in \mathcal{X}_{\text{reg}}$, we have

$$(T\bar{J})(x) = \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} \bar{J}(x') \tau(x'|x, u) \right\} \quad (3.13a)$$

$$\leq \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} (J(x') + d) \tau(x'|x, u) \right\} \quad (3.13b)$$

$$= \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J(x') \tau(x'|x, u) \right\} + \gamma d \quad (3.13c)$$

$$= (TJ)(x) + \gamma d \quad (3.13d)$$

where (3.13b) follows from the second inequality of (3.10). Similarly, from the first inequality of (3.10), we obtain $(TJ)(x) - \gamma d \leq (T\bar{J})(x)$. Therefore, we have

$$|(TJ)(x) - (T\bar{J})(x)| \leq \gamma d \quad \forall x \in \mathcal{X}_{\text{reg}}. \quad (3.14)$$

From (3.12) and (3.14), we obtain $\|TJ - T\bar{J}\|_\infty \leq \gamma d$ as desired. $\blacksquare$

The contraction property of T has several important implications. First, by the Banach fixed-point theorem,¹ it implies that T has a unique fixed point J^* such that $TJ^* = J^*$. Second, for any J_0 , the sequence defined by $J_{k+1} = TJ_k$ satisfies $\lim_{k \rightarrow \infty} J_k = J^*$. It can also be shown that the convergence is linear, i.e., the error $\|J_k - J^*\|_\infty$ diminishes like γ^k (Exercise 3.1). Moreover, by Theorem 2.3, the unique fixed point J^* must coincide with the value function of (3.6). Finally, since $\|J_{k+1} - J_k\|_\infty$ is also a convergent sequence, the condition $\|J_{k+1} - J_k\|_\infty \leq \epsilon$ can be used as a stopping criterion for the value iteration algorithm, as shown in Algorithm 6.

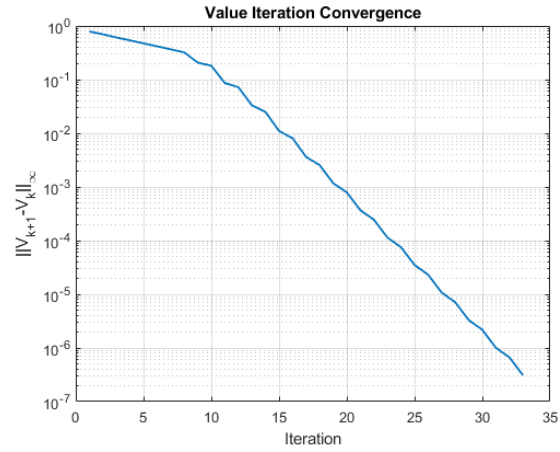
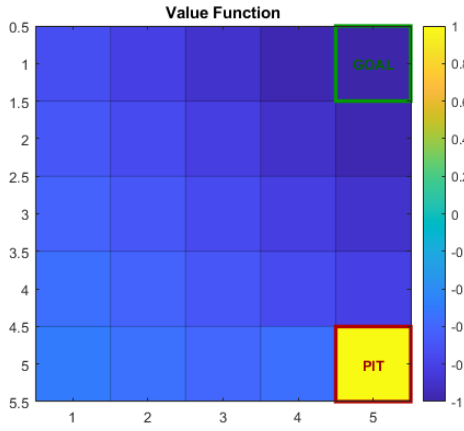
¹Let $(\mathcal{X}, d)$ be a complete metric space. Let $T : \mathcal{X} \rightarrow \mathcal{X}$ be a contraction mapping, i.e., there exists a constant $\gamma \in (0, 1)$ such that $d(T(x_1), T(x_2)) \leq \gamma d(x_1, x_2)$ for all $x_1, x_2 \in \mathcal{X}$. Then T admits a unique fixed point $x^* \in \mathcal{X}$ such that $T(x^*) = x^*$.

Algorithm 6: Value Iteration (Finite MDP, discounted cost, variable end-time)

Input: Finite MDP $(\mathcal{X}_{\text{reg}} \cup \mathcal{X}_{\text{abs}}, \mathcal{U}, \tau, C, \gamma)$ with regular states $\mathcal{X}_{\text{reg}}$ and absorbing states $\mathcal{X}_{\text{abs}}$, discount factor $\gamma \in (0, 1)$, and a variable end-time (3.4).

- 1 Initialize $J(x)$ arbitrarily for each $x \in \mathcal{X}_{\text{reg}}$;
- 2 Initialize $J(x) = C^{\text{exit}}(x)$ for each $x \in \mathcal{X}_{\text{abs}}$;
- 3 **do**
- 4 $J_{\text{old}} \leftarrow J$;
- 5 $J(x) \leftarrow \min_u \{C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J_{\text{old}}(x') \tau(x'|x, u)\}$ for each $x \in \mathcal{X}_{\text{reg}}$;
- 6 **while** $\|J - J_{\text{old}}\|_{\infty} > \epsilon$;

Output: J



Value function obtained after 35 value iterations.

Convergence of the value iteration.

Figure 3.3: Output of Algorithm 6 applied to the gridworld MDP in Example 3.1.

Exercise 3.1 (i) Show that $\|J_k - J^*\|_{\infty} \leq \gamma^k \|J_0 - J^*\|$.

(ii) Show that the stopping condition $\|J_{k+1} - J_k\|_{\infty} \leq \epsilon$ implies

$$\|J_{k+1} - J^*\|_{\infty} \leq \frac{\epsilon\gamma}{1-\gamma}. \quad (3.15)$$

Exercise 3.2 Consider the gridworld setup in Example 3.1 and assume $c_{\text{pit}} = +1$, $c_{\text{goal}} = -1$, $c_{\text{step}} = 0.05$ and $p = 0.2$.

(i) Using Algorithm 6, compute the value function. Hint: You should obtain a value function as shown in Figure 3.3.

(ii) Compute the optimal policy. Display the optimal action at each state by showing an arrow in each cell.

3.3 Simulator-Driven RL: Monte Carlo Method

Although value iteration is simple and forms the foundation of many reinforcement learning algorithms, it is often impractical in real-world scenarios because it relies on explicit model information, which may not be readily available. In particular, the minimization step in line 5 of Algorithm 6 assumes that the function to be minimized is explicitly known. This requires knowledge of the cost function $C(x, u)$ and the transition kernel $\tau(x'|x, u)$, which are often unavailable in practice.

Even so, it is sometimes straightforward to simulate a full episode $x_0, u_0, x_1, u_1, \dots$ under a given policy μ . As emphasized in [1], *“in surprisingly many cases, it is easy to generate experience sampled according to the desired probability distributions, but infeasible to obtain the distributions in explicit form.”* Although our focus here is on finite MDPs, this observation extends to continuous state-action problems as well. For instance, trained artificial neural networks (ANNs) can often be used to efficiently simulate $C(x, u)$ and $\tau(x'|x, u)$ on continuous variables. However, Algorithm 6 is not in the most convenient form to exploit such simulators.

This section introduces Monte Carlo approaches to reinforcement learning, which are well suited to cases where explicit models are unavailable, but generating full episodes $x_0, u_0, x_1, u_1, \dots$ from the correct distribution is computationally inexpensive. As we will see, the path integral control method, which will be covered in Chapter 8, is also motivated by such scenarios.

3.3.1 Q-Function

Even if the optimal value function $J^*(x)$ were known, it may not be straightforward to recover the optimal policy via

$$\mu^*(x) = \arg \min_{u \in \mathcal{U}} \left\{ C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J^*(x') \tau(x'|x, u) \right\}, \quad (3.16)$$

since this requires explicit knowledge of $C(x, u)$ and $\tau(x'|x, u)$. To avoid this difficulty, model-free methods typically use the *Q-function* rather than the value function.

Unlike the value function $J(x)$, which assigns a value to each state x , the Q-function assigns a value to each state-action pair (x, u) :²

$$Q^*(x, u) := C(x, u) + \gamma \sum_{x' \in \mathcal{X}} J^*(x') \tau(x'|x, u). \quad (3.17)$$

The value function can be recovered from the Q-function as

$$J^*(x) = \min_{u \in \mathcal{U}} Q^*(x, u), \quad (3.18)$$

and the optimal policy is then implemented simply as

$$\mu^*(x) = \arg \min_{u \in \mathcal{U}} Q^*(x, u). \quad (3.19)$$

The Monte Carlo method estimates the Q-function by simulating episodes $x_0, u_0, x_1, u_1, \dots$ and computing empirical averages of the costs.

²For this reason, the Q-function is also known as the state-action value function.

3.3.2 Monte Carlo ES: Discounted Problems

To estimate $Q^*(x, u)$ for every state-action pair (x, u) using Monte Carlo simulations, it is necessary that each pair is sampled infinitely often in the simulated episodes. To ensure this, the Monte Carlo ES (“exploring starts”) algorithm generates samples starting from all possible initial state-action pairs. We present below a variant of Monte Carlo ES studied by Tsitsiklis [2].

The algorithm of [2] considers the infinite-horizon discounted cost problem:

$$\min_{\mu_{0:\infty}} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t C(x_t, u_t) \right], \quad (3.20)$$

and its pseudocode description is shown in Algorithm 7. Notice that Algorithm 7 selects all possible state-action pairs as the initial condition. Tsitsiklis [2] proved that, under this scheme, $Q(x, u)$ converges to $Q^*(x, u)$ with probability one for each (x, u) , provided that the step-size sequence $\{\alpha_k\}$ satisfies the *Robbins-Monro* conditions:

$$\sum_{k=0}^{\infty} \alpha_k = \infty, \quad \sum_{k=0}^{\infty} \alpha_k^2 < \infty. \quad (3.21)$$

Algorithm 7: Monte Carlo ES [2]

Input: Infinite-horizon discounted MDP (3.20), step-size sequence α_k satisfying (3.21)

- 1 Initialize $\mu(x)$ and $Q(x, u)$ arbitrarily for all $x \in \mathcal{X}, u \in \mathcal{U}$;
- 2 **for** $k = 1, 2, \dots$ **do**
- 3 **for each state-action pair** $(x_0, u_0) \in \mathcal{X} \times \mathcal{U}$ **do**
- 4 Generate an episode $x_0, u_0, x_1, u_1, x_2, u_2, \dots$ following μ starting from (x_0, u_0) ;
- 5 Compute the discounted return $\bar{Q} = \sum_{t=0}^{\infty} \gamma^t C(x_t, u_t)$;
- 6 Update $Q(x_0, u_0) \leftarrow (1 - \alpha_k)Q(x_0, u_0) + \alpha_k \bar{Q}$;
- 7 **end**
- 8 Update policy: $\mu(x) \leftarrow \arg \min_u Q(x, u), \quad \forall x \in \mathcal{X}$;
- 9 **end**

Output: μ, Q

Lines 4-5 of Algorithm 7 seemingly require infinitely long simulated episodes to compute the discounted cost. As a practical remedy, Tsitsiklis [2] proposed terminating the simulation at each time step with probability $1 - \gamma$. This guarantees that episodes end in finite time with probability one, and the cost can then be computed as an *undiscounted* sum. The expected undiscounted cost until termination is equal to the expected discounted cost in the infinite-horizon setting, which justifies this modification.

Although Algorithm 7 converges for general discounted-cost problems, its convergence relies critically on several features of the algorithm. First, the uniform sampling of the initial state-action pairs (x_0, u_0) is essential. Second, each simulated episode is used to update only a single component of the Q-function, namely $Q(x_0, u_0)$. This appears inefficient, since many state-action pairs are typically visited in a single episode. However, updating multiple Q-values from a single episode while still guaranteeing convergence to Q^* is nontrivial and requires additional structural assumptions on the underlying MDP.

3.3.3 Monte Carlo ES: Undiscounted Problems

The second variant of the Monte Carlo ES algorithm, called the *first-visit Monte Carlo ES*, addresses the class of variable end-time problems of the form

$$\min_{\mu_{0:\infty}} \mathbb{E} \left[\sum_{t=0}^{T_f-1} C(x_t, u_t) \right]. \quad (3.22)$$

We assume that the underlying MDP terminates at a finite end-time T_f with probability one (recall Assumption 2.1). For simplicity, we further assume that the exit cost is zero, i.e., $C^{\text{exit}}(x_{T_f}) = 0$.

Algorithm 8 was introduced in [1], and its convergence was analyzed in [3]. This algorithm differs from Algorithm 7 in several respects. First, it does not require initial state-action pairs to be sampled uniformly; it only requires that each pair be sampled infinitely often. Second, a single simulated episode can update multiple components of the Q-function. This addresses the inefficiency issue of Algorithm 7.

Algorithm 8: First-visit Monte Carlo ES [1, 3]

Input: A finite-state MDP (3.22), a PMF on $\mathcal{X} \times \mathcal{U}$ such that $p(x, u) > 0$ for all (x, u) , and an upper bound M on $|\mathcal{X}|$.

```

1 Initialize  $\mu(x)$  arbitrarily and  $Q(x, u) = -\infty$  for all  $x \in \mathcal{X}, u \in \mathcal{U}$ ;
2 Initialize  $\text{Values}(x, u)$  as an empty list for all  $x \in \mathcal{X}, u \in \mathcal{U}$ ;
3 for  $k = 1, 2, \dots$  do
4   Sample  $(x_0, u_0) \sim p(x, u)$ ;
5   Generate an episode  $x_0, u_0, x_1, u_1, \dots, u_{T_f-1}, x_{T_f}$  following  $\mu$ , starting from  $(x_0, u_0)$ ;
6   if  $T_f > M$  then
7     Terminate the episode at step  $M$ ;
8   end
9    $G \leftarrow 0$ ;
10  for  $t = T_f - 1, T_f - 2, \dots, 0$  do
11     $G \leftarrow G + C(x_t, u_t)$ ;
12    if  $(x_t, u_t)$  does not appear in  $x_0, u_0, \dots, x_{t-1}, u_{t-1}$  then
13      Append  $G$  to  $\text{Values}(x_t, u_t)$ ;
14       $Q(x_t, u_t) \leftarrow \text{average}(\text{Values}(x_t, u_t))$ ;
15    end
16  end
17   $\mu(x) \leftarrow \arg \min_u Q(x, u), \quad \forall x \in \mathcal{X}$ ;
18 end

```

Output: μ, Q

Despite these improvements, the convergence of Algorithm 8 to Q^* is guaranteed only under somewhat restrictive conditions. The following result is due to [3]:

Definition 3.1 A finite-state MDP is said to be Optimal Policy Feed-Forward (OPFF) if, under any optimal policy, no state is ever revisited.

Theorem 3.1 Suppose the finite-state MDP is OPFF. Then $Q(x, u)$ in Algorithm 8 converges to $Q^*(x, u)$ for each $(x, u) \in \mathcal{X} \times \mathcal{U}$ with probability one.

3.4 Data-driven RL: Temporal difference method

Temporal difference (TD) learning is similar to Monte Carlo methods in that it does not require explicit knowledge of the underlying MDP. However, unlike Monte Carlo methods, TD learning allows the Q-function to be updated at every time step without waiting until the end of the simulation of the full episode. This is a convenient feature for data-driven synthesis, where the policy is updated as soon as new data (from the environment) arrives. The TD counterpart of Algorithm 7 is Q-Learning, which is summarized as Algorithm 9.

Algorithm 9: Q-learning

Input: Small $\epsilon > 0$ and step sizes $\alpha_t(x, u)$ for each $t = 0, 1, \dots$ and $(x, u) \in \mathcal{X} \times \mathcal{U}$.

- 1 Initialize $\mu(x)$ and $Q(x, u)$ for all $x \in \mathcal{X}$ and $u \in \mathcal{U}$;
- 2 Initialize $x_0 \in \mathcal{X}$;
- 3 **for** $t = 0, 1, 2, \dots$ **do**
- 4 Choose $u_t = \mu^\epsilon(x_t)$ (ϵ -greedy policy);
- 5 Take action u_t , and observe $C_t = C(x_t, u_t)$ and x_{t+1} ;
- 6 Set $Q(x_t, u_t) \leftarrow (1 - \alpha_t(x_t, u_t))Q(x_t, u_t) + \alpha_t(x_t, u_t) [C_t + \gamma \min_u Q(x_{t+1}, u)]$;
- 7 $\mu(x) \leftarrow \arg \min_u Q(x, u), \quad \forall x \in \mathcal{X}$;
- 8 **end**

Output: μ, Q

The convergence of Q-learning was first established by Watkins [4, 5]. An alternative proof was later provided by Tsitsiklis [6] under more general conditions. The convergence requirements for Q-learning are relatively mild compared to those for Monte Carlo ES, which is another reason Q-learning is considered attractive. Tsitsiklis employed the method of stochastic approximation to establish the convergence result, summarized below.

3.4.1 Stochastic approximation

Consider the problem of finding a vector $\theta = \theta^* \in \mathbb{R}^n$ satisfying

$$f(\theta) := \mathbb{E}^w[F(\theta, w)] = 0, \quad (3.23)$$

when no analytical expression of the function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is available. We assume, however, that experiments are possible: for each test point $\theta(i)$, we obtain a noisy observation $y(i) = F(\theta(i), w(i))$. Suppose we know that the mean of $y(i)$ equals $f(\theta(i))$, even though the noise random variable $w(i)$ is not observable. To ensure that the problem is well posed, we assume that $f(\theta)$ is a nondecreasing function such that $\frac{d}{d\theta}f(\theta^*)$ exists and is positive.

The stochastic approximation algorithm of Robbins and Monro [7] attempts to compute θ^* by recursively updating the estimate $\theta(t)$ according to

$$\begin{aligned} \theta(t+1) &= \theta(t) - \alpha(t)y(t) \\ &= \theta(t) - \alpha(t)F(\theta(t), w(t)). \end{aligned} \quad (3.24)$$

Robbins and Monro [7] showed that (3.24) converges to θ^* in L^2 (and therefore in probability) if $F(\theta, w)$ is uniformly bounded for each θ , and if the step sizes satisfy

$$\sum_{t=0}^{\infty} \alpha(t) = \infty, \quad \sum_{t=0}^{\infty} \alpha^2(t) < \infty. \quad (3.25)$$

Blum [8] later established almost sure convergence.

The scope of stochastic approximation theory is extremely broad, as illustrated below.

Example 3.2 (Empirical mean) *Given i.i.d. realizations $w(0), w(1), \dots, w(t-1)$, the MSE estimate of $\mathbb{E}[w]$ is*

$$\theta(t) = \frac{1}{t} \sum_{i=0}^{t-1} w(i).$$

Writing this in recursive form, we obtain

$$\begin{aligned} \theta(t+1) &= \frac{t}{t+1} \theta(t) + \frac{1}{t+1} w(t) \\ &= \theta(t) - \frac{1}{t+1} (\theta(t) - w(t)). \end{aligned} \quad (3.26)$$

We can view (3.26) as a stochastic approximation with $\alpha(t) = \frac{1}{t+1}$ and $F(\theta, w) = \theta - w$. By the strong law of large numbers, $\theta(t) \rightarrow \mathbb{E}[w]$ almost surely.

Example 3.3 (Stochastic gradient descent) *Suppose that at each $\theta \in \mathbb{R}^n$, we can observe the noisy gradient $y = \nabla h(\theta) + w$ of a function h , where w is a zero-mean random variable. The stochastic approximation (3.24) becomes*

$$\theta(t+1) = \theta(t) - \alpha(t) (\nabla h(\theta) + w), \quad (3.27)$$

which is known as the stochastic gradient descent algorithm.

Example 3.4 (Fixed-point iteration) *Suppose we want to compute a fixed point θ^* of a map T satisfying $\theta^* = T(\theta^*)$. Assume that for each θ , we observe $y = T(\theta) + w$ where w is a zero-mean noise. Rather than the simple iteration $\theta(t+1) = T(\theta(t)) + w(t)$ (which does not converge), consider the moving average*

$$\begin{aligned} \theta(t+1) &= (1 - \alpha(t)) \theta(t) + \alpha(t) (T(\theta(t)) + w(t)) \\ &= \theta(t) - \alpha(t) [\theta(t) - T(\theta(t)) - w(t)]. \end{aligned} \quad (3.28)$$

This is a stochastic approximation algorithm with $F(\theta, w) = \theta - T(\theta) - w$.

3.4.2 Convergence of Q-Learning

The convergence proof of Tsitsiklis [6] exploits the connection between Q-learning and the stochastic approximation algorithm highlighted in Example 3.4. Specifically, one can define a contraction mapping for the Q-function in a similar manner to the contraction mapping for the value function.

Exercise 3.3 For each function $Q : \mathcal{X} \times \mathcal{U} \rightarrow \mathbb{R}$, define the operator T by

$$(TQ)(x, u) = C(x, u) + \gamma \sum_{x' \in \mathcal{X}} \tau(x'|x, u) \min_{u' \in \mathcal{U}} Q(x', u'), \quad \forall x \in \mathcal{X}, u \in \mathcal{U}, \quad (3.29)$$

where $\gamma \in (0, 1)$. Introducing the maximum norm $\|Q\|_\infty := \max_{x,u} |Q(x, u)|$, show that

$$\|TQ_1 - TQ_2\|_\infty \leq \gamma \|Q_1 - Q_2\|_\infty \quad (3.30)$$

for all $Q_1, Q_2 : \mathcal{X} \times \mathcal{U} \rightarrow \mathbb{R}$. Discuss the existence and uniqueness of a fixed point Q^* satisfying $T(Q^*) = Q^*$. Does the iteration $Q(t+1) = T(Q(t))$ converge?

In data-driven scenarios, the exact fixed-point iteration $Q(t+1) = T(Q(t))$ is infeasible. Instead, we can only perform a noisy fixed-point iteration as in Example 3.4. Moreover, as shown in Algorithm 9, the different components of the Q-function must be updated asynchronously.

This motivates the study of the following asynchronous stochastic approximation algorithm. Let $\theta = (\theta_1, \dots, \theta_n) \in \mathbb{R}^n$ and $T : \mathbb{R}^n \rightarrow \mathbb{R}^n$ a mapping with components $T(\theta) = (T_1(\theta), \dots, T_n(\theta))$, where each $T_i : \mathbb{R}^n \rightarrow \mathbb{R}$ is a component mapping. Let $\mathbb{N}^i \subset \mathbb{N}$ be the set of time indices at which the component θ_i is updated. For each i , the update rule is given by

$$\theta_i(t+1) = \theta_i(t) - \alpha_i(t) [\theta_i(t) - T_i(\theta(t)) - w_i(t)], \quad (3.31)$$

where $\alpha_i(t) \in (0, 1)$ for $t \in \mathbb{N}^i$ and $\alpha_i(t) = 0$ for $t \notin \mathbb{N}^i$.

We allow $\theta(0)$, $w_i(t)$, and $\alpha_i(t)$ to be random variables on the probability space $(\Omega, \mathcal{F}, \mathcal{P})$, and assume the following conditions hold for an increasing sequence of σ -subalgebras $\mathcal{F}(0) \subset \mathcal{F}(1) \subset \dots \subset \mathcal{F}$:

Assumption 3.1 • $\theta(0)$ is $\mathcal{F}(0)$ -measurable.

- $w_i(t)$ is $\mathcal{F}(t+1)$ -measurable for each i and t .
- $\mathbb{E}[w_i(t) \mid \mathcal{F}(t)] = 0$ for each i and t .
- There exist deterministic constants $A, B < \infty$ such that

$$\mathbb{E}[w_i^2(t) \mid \mathcal{F}(t)] \leq A + B \max_j \max_{\tau \leq t} |\theta_j(\tau)|^2, \quad \forall i, t. \quad (3.32)$$

- $\alpha_i(t)$ is $\mathcal{F}(t)$ -measurable for each i and t .

The following result applies to the asynchronous stochastic approximation algorithm (3.31):

Theorem 3.2 Suppose Assumption 3.1 holds for (3.31), and assume that each $\mathbb{N}^i$ is infinite.³ For each i , suppose there exists a constant $C < \infty$ such that

$$\sum_{t=0}^{\infty} \alpha_i(t) = \infty, \quad \sum_{t=0}^{\infty} \alpha_i^2(t) < C \quad (3.33)$$

with probability one. Finally, assume there exists a vector $\theta^* \in \mathbb{R}^n$ such that

$$\|T(\theta) - \theta^*\|_\infty \leq \gamma \|\theta - \theta^*\|_\infty, \quad \forall \theta \in \mathbb{R}^n, \quad (3.34)$$

for some $\gamma \in (0, 1)$. Then $\theta(t)$ converges to θ^* with probability one.

³This ensures that each coordinate θ_i is updated infinitely often.

Convergence of Algorithm 9 follows directly from Theorem 3.2 by identifying Q with θ . In summary, Algorithm 9 converges to Q^* with probability one if the step sizes are chosen to satisfy

$$\sum_{t \in \mathbb{N}^{x,u}} \alpha_t(x, u) = \infty, \quad \sum_{t \in \mathbb{N}^{x,u}} \alpha_t^2(x, u) < \infty, \quad (3.35)$$

where $\mathbb{N}^{x,u}$ is the infinite set of time indices at which $Q(x, u)$ is updated.

To ensure that all pairs (x, u) are visited infinitely often, it is typically necessary to randomize the behavior policy (i.e., the policy used during data collection). For example, Algorithm 9 adopts the so-called ϵ -greedy policy:

$$\mu^\epsilon(x) = \begin{cases} \mu(x), & \text{with probability } 1 - \epsilon, \\ u \sim \text{Unif}(\mathcal{U}), & \text{with probability } \epsilon, \end{cases}$$

as shown in line 4. The nonzero probability of selecting a random action guarantees sufficient exploration. Importantly, this does not prevent the target policy (the policy being learned) from converging to the optimal policy μ^* . Indeed, Q-learning is an *off-policy* TD method in the sense that the behavior policy and the target policy may differ. This is another advantage of Q-learning.

3.4.3 Monte Carlo vs. TD Learning

We have seen that TD learning allows the Q-function to be updated at every time step, without waiting until the end of an episode. This is a significant advantage over Monte Carlo methods. We have also noted that the theoretical convergence proof of Q-learning is more robust, in that it does not require the restrictive assumptions on the underlying MDPs that Monte Carlo ES does (c.f., Algorithm 8). These observations naturally raise the question: under what circumstances might Monte Carlo methods still be preferred over TD methods?

In fact, there are several practical reasons why Monte Carlo methods are sometimes preferred. First, they are conceptually simple and easy to implement. Second, they do not involve *bootstrapping* operations.⁴ Bootstrapping, combined with off-policy training, often introduces instability in practice. (Bootstrapping, off-policy training, and function approximation are collectively referred to as the *deadly triad* [1].) Finally, TD methods are generally formulated for time-invariant, infinite-horizon problems, whereas Monte Carlo methods can be directly applied to time-varying and finite-horizon problems.

⁴Bootstrapping refers to using a previous estimate of the value function (or Q-function) to generate new data for the current estimate. This operation is also called a *Bellman backup*.

Bibliography

- [1] R. S. Sutton and A. G. Barto, “Reinforcement learning: An introduction,” 2018.
- [2] J. N. Tsitsiklis, “On the convergence of optimistic policy iteration,” *Journal of Machine Learning Research*, vol. 3, no. Jul, pp. 59–72, 2002.
- [3] C. Wang, S. Yuan, K. Shao, and K. W. Ross, “On the convergence of the monte carlo exploring starts algorithm for reinforcement learning,” in *International Conference on Learning Representations*, 2022.
- [4] C. J. C. H. Watkins *et al.*, “Learning from delayed rewards,” 1989.
- [5] C. J. Watkins and P. Dayan, “Q-learning,” *Machine learning*, vol. 8, no. 3, pp. 279–292, 1992.
- [6] J. N. Tsitsiklis, “Asynchronous stochastic approximation and q-learning,” *Machine learning*, vol. 16, no. 3, pp. 185–202, 1994.
- [7] H. Robbins and S. Monro, “A stochastic approximation method,” *The annals of mathematical statistics*, pp. 400–407, 1951.
- [8] J. R. Blum, “Approximation methods which converge with probability one,” *The Annals of Mathematical Statistics*, pp. 382–386, 1954.